

MEDIGUARD PROJECT Part 2

Complete Implementation Guide

Secure Federated Health Intelligence Platform

A Privacy-Preserving Machine Learning System for Healthcare

Contents

1	PHASE 3: Cryptography Layer	2
1.1	Public Key Infrastructure (PKI) Setup	2
1.1.1	Create PKI Directory Structure	2
1.1.2	Create OpenSSL Configuration	2
1.1.3	Generate Root CA (Master Key — Keep Offline)	4
1.1.4	Create Intermediate CA Configuration	5
1.1.5	Generate Intermediate CA	7
1.1.6	Generate Node Certificates	8
1.1.7	Distribute Certificates to Nodes	10
1.2	AES-256-GCM Encryption Implementation	11
1.2.1	Create Encryption Module	11
1.3	Database Integration with Encryption	17
1.4	Digital Signatures Implementation	20
1.5	Zero-Knowledge Proofs Implementation	23
1.6	Automated Key Rotation	29
1.7	Phase 3 Verification Checklist	30
1.8	Troubleshooting	31

Chapter 1

PHASE 3: Cryptography Layer

Phase Details

Duration: Weeks 4–5

Goal: Implement enterprise-grade encryption, PKI, digital signatures, and zero-knowledge proofs

1.1 Public Key Infrastructure (PKI) Setup

1.1.1 Create PKI Directory Structure

On `ml-server` VM (acts as Certificate Authority):

```
1  # Create PKI directory structure
2  sudo mkdir -p /opt/mediguard-pki/{root-ca,intermediate-ca,certs
   ,private,csr}
3  cd /opt/mediguard-pki
4
5  # Set secure permissions
6  sudo chmod 700 private
7  sudo chmod 755 certs csr
8
9  # Create index and serial files
10 sudo touch root-ca/index.txt
11 sudo echo "1000" | sudo tee root-ca/serial
12 sudo touch intermediate-ca/index.txt
13 sudo echo "2000" | sudo tee intermediate-ca/serial
```

1.1.2 Create OpenSSL Configuration

Create file: `/opt/mediguard-pki/openssl-root.cnf`

```
1  sudo vim /opt/mediguard-pki/openssl-root.cnf
```

```
1  # OpenSSL Root CA Configuration
2  [ ca ]
3  default_ca = CA_default
4
5  [ CA_default ]
6  dir                = /opt/mediguard-pki/root-ca
7  certs              = /opt/mediguard-pki/certs
8  crl_dir            = /opt/mediguard-pki/root-ca/crl
9  new_certs_dir      = /opt/mediguard-pki/certs
10 database           = /opt/mediguard-pki/root-ca/index.txt
11 serial             = /opt/mediguard-pki/root-ca/serial
12 RANDFILE           = /opt/mediguard-pki/private/.rand
13
14 private_key        = /opt/mediguard-pki/private/root-ca.key
15 certificate         = /opt/mediguard-pki/certs/root-ca.crt
16
17 crlnumber           = /opt/mediguard-pki/root-ca/crlnumber
18 crl                 = /opt/mediguard-pki/root-ca/crl/ca.crl
19 crl_extensions     = crl_ext
20 default_crl_days   = 30
21
22 default_md         = sha256
23 name_opt           = ca_default
24 cert_opt           = ca_default
25 default_days       = 3650
26 preserve           = no
27 policy             = policy_strict
28
29 [ policy_strict ]
30 countryName        = match
31 stateOrProvinceName = match
32 organizationName   = match
33 organizationalUnitName = optional
34 commonName         = supplied
35 emailAddress       = optional
36
37 [ req ]
38 default_bits       = 4096
39 distinguished_name = req_distinguished_name
40 string_mask        = utf8only
41 default_md         = sha256
42 x509_extensions    = v3_ca
43
44 [ req_distinguished_name ]
45 countryName        = Country Name (2 letter code)
46 stateOrProvinceName = State or Province Name
47 localityName       = Locality Name
48 organizationName   = Organization Name
49 organizationalUnitName = Organizational Unit Name
```

```

50 commonName          = Common Name
51 emailAddress         = Email Address
52
53 countryName_default  = US
54 stateOrProvinceName_default = California
55 localityName_default = San Francisco
56 0.organizationName_default = MediGuard Health Systems
57 organizationalUnitName_default = IT Security
58 emailAddress_default  = security@mediguard.local
59
60 [ v3_ca ]
61 subjectKeyIdentifier = hash
62 authorityKeyIdentifier = keyid:always,issuer
63 basicConstraints      = critical, CA:true
64 keyUsage              = critical, digitalSignature, cRLSign,
    keyCertSign
65
66 [ v3_intermediate_ca ]
67 subjectKeyIdentifier = hash
68 authorityKeyIdentifier = keyid:always,issuer
69 basicConstraints      = critical, CA:true, pathlen:0
70 keyUsage              = critical, digitalSignature, cRLSign,
    keyCertSign
71
72 [ server_cert ]
73 basicConstraints      = CA:FALSE
74 nsCertType           = server
75 nsComment             = "MediGuard_Server_Certificate"
76 subjectKeyIdentifier = hash
77 authorityKeyIdentifier = keyid,issuer:always
78 keyUsage              = critical, digitalSignature,
    keyEncipherment
79 extendedKeyUsage      = serverAuth
80
81 [ crl_ext ]
82 authorityKeyIdentifier = keyid:always

```

1.1.3 Generate Root CA (Master Key — Keep Offline)

```

1 cd /opt/mediguard-pki
2
3 # Generate Root CA private key (4096-bit RSA)
4 sudo openssl genrsa -aes256 -out private/root-ca.key 4096
5
6 # IMPORTANT: Choose a STRONG passphrase and SAVE IT SECURELY
7 # Example passphrase: "MediGuard-Root-CA-2024!@#$$%"
8
9 # Set strict permissions on private key
10 sudo chmod 400 private/root-ca.key

```

```
11
12 # Generate Root CA certificate (valid 10 years)
13 sudo openssl req -config openssl-root.cnf \
14   -key private/root-ca.key \
15   -new -x509 -days 3650 -sha256 \
16   -extensions v3_ca \
17   -out certs/root-ca.crt
18
19 # Enter certificate details when prompted:
20 # Country Name: US
21 # State: California
22 # Locality: San Francisco
23 # Organization: MediGuard Health Systems
24 # Organizational Unit: Root Certificate Authority
25 # Common Name: MediGuard Root CA
26 # Email: root-ca@mediguard.local
27
28 # Verify Root CA certificate
29 sudo openssl x509 -noout -text -in certs/root-ca.crt
30
31 # Display certificate summary
32 echo "=== ROOT CA CERTIFICATE ==="
33 sudo openssl x509 -noout -subject -issuer -dates -in certs/root
   -ca.crt
```

Expected output:

```
subject=C = US, ST = California, L = San Francisco,
         O = MediGuard Health Systems,
         OU = Root Certificate Authority, CN = MediGuard Root CA
issuer=C = US, ST = California, L = San Francisco,
        O = MediGuard Health Systems,
        OU = Root Certificate Authority, CN = MediGuard Root CA
notBefore=Jan 15 10:00:00 2024 GMT
notAfter=Jan 12 10:00:00 2034 GMT
```

1.1.4 Create Intermediate CA Configuration

Create file: /opt/mediguard-pki/openssl-intermediate.cnf

```
1 sudo vim /opt/mediguard-pki/openssl-intermediate.cnf
```

```
1 # OpenSSL Intermediate CA Configuration
2 [ ca ]
3 default_ca = CA_default
4
5 [ CA_default ]
6 dir           = /opt/mediguard-pki/intermediate-ca
7 certs         = /opt/mediguard-pki/certs
```

```

8  crl_dir           = /opt/mediguard-pki/intermediate-ca/crl
9  new_certs_dir     = /opt/mediguard-pki/certs
10 database          = /opt/mediguard-pki/intermediate-ca/index.
    txt
11 serial            = /opt/mediguard-pki/intermediate-ca/serial
12 RANDFILE          = /opt/mediguard-pki/private/.rand
13
14 private_key        = /opt/mediguard-pki/private/intermediate-ca.
    key
15 certificate        = /opt/mediguard-pki/certs/intermediate-ca.
    crt
16
17 crlnumber          = /opt/mediguard-pki/intermediate-ca/
    crlnumber
18 crl                = /opt/mediguard-pki/intermediate-ca/crl/
    intermediate-ca.crl
19 crl_extensions     = crl_ext
20 default_crl_days   = 30
21
22 default_md         = sha256
23 name_opt           = ca_default
24 cert_opt           = ca_default
25 default_days       = 375
26 preserve           = no
27 policy             = policy_loose
28
29 [ policy_loose ]
30 countryName        = optional
31 stateOrProvinceName = optional
32 localityName       = optional
33 organizationName    = optional
34 organizationalUnitName = optional
35 commonName          = supplied
36 emailAddress        = optional
37
38 [ req ]
39 default_bits        = 2048
40 distinguished_name   = req_distinguished_name
41 string_mask          = utf8only
42 default_md          = sha256
43 x509_extensions     = v3_ca
44
45 [ req_distinguished_name ]
46 countryName         = Country Name (2 letter code)
47 stateOrProvinceName = State or Province Name
48 localityName        = Locality Name
49 O.organizationName   = Organization Name
50 organizationalUnitName = Organizational Unit Name
51 commonName           = Common Name

```

```

52 emailAddress          = Email Address
53
54 [ v3_ca ]
55 subjectKeyIdentifier   = hash
56 authorityKeyIdentifier = keyid:always,issuer
57 basicConstraints       = critical, CA:true
58 keyUsage               = critical, digitalSignature, cRLSign,
    keyCertSign
59
60 [ v3_intermediate_ca ]
61 subjectKeyIdentifier   = hash
62 authorityKeyIdentifier = keyid:always,issuer
63 basicConstraints       = critical, CA:true, pathlen:0
64 keyUsage               = critical, digitalSignature, cRLSign,
    keyCertSign
65
66 [ server_cert ]
67 basicConstraints       = CA:FALSE
68 nsCertType             = server
69 nsComment               = "MediGuard_Server_Certificate"
70 subjectKeyIdentifier   = hash
71 authorityKeyIdentifier = keyid,issuer:always
72 keyUsage               = critical, digitalSignature,
    keyEncipherment
73 extendedKeyUsage       = serverAuth
74 subjectAltName          = @alt_names
75
76 [ alt_names ]
77 DNS.1 = *.mediguard.local
78 IP.1  = 10.0.0.1
79 IP.2  = 10.0.0.2
80 IP.3  = 10.0.0.3
81
82 [ crl_ext ]
83 authorityKeyIdentifier = keyid:always

```

1.1.5 Generate Intermediate CA

```

1 cd /opt/mediguard-pki
2
3 # Generate Intermediate CA private key
4 sudo openssl genrsa -aes256 -out private/intermediate-ca.key
    4096
5
6 # Choose passphrase: "MediGuard-Intermediate-CA-2024!@#"
7 sudo chmod 400 private/intermediate-ca.key
8
9 # Generate Certificate Signing Request (CSR)
10 sudo openssl req -config openssl-intermediate.cnf \

```



```

11  -new -sha256 \
12  -key private/intermediate-ca.key \
13  -out csr/intermediate-ca.csr
14
15  # Enter details:
16  # Common Name: MediGuard Intermediate CA
17
18  # Sign Intermediate CA with Root CA
19  sudo openssl ca -config openssl-root.cnf \
20  -extensions v3_intermediate_ca \
21  -days 1825 -notext -md sha256 \
22  -in csr/intermediate-ca.csr \
23  -out certs/intermediate-ca.crt
24
25  # Enter Root CA passphrase when prompted
26  # Type 'y' to sign and commit
27
28  # Verify Intermediate CA
29  sudo openssl x509 -noout -text -in certs/intermediate-ca.crt
30
31  # Create certificate chain file
32  sudo cat certs/intermediate-ca.crt certs/root-ca.crt | \
33  sudo tee certs/ca-chain.crt

```

1.1.6 Generate Node Certificates

Create certificate generation script:

```
1 sudo vim /opt/mediguard-pki/generate-node-cert.sh
```

```

1  #!/bin/bash
2  # Generate Node Certificate for MediGuard VMs
3
4  if [ $# -ne 2 ]; then
5      echo "Usage: $0 <node-name> <node-ip>"
6      echo "Example: $0 hospital-a 10.0.0.1"
7      exit 1
8  fi
9
10  NODE_NAME=$1
11  NODE_IP=$2
12  PKI_DIR="/opt/mediguard-pki"
13
14  cd $PKI_DIR
15
16  # Generate node private key (no passphrase for automated services)
17  echo "Generating private key for ${NODE_NAME}..."
18  openssl genrsa -out private/${NODE_NAME}.key 2048
19  chmod 400 private/${NODE_NAME}.key
20
21  # Create CSR configuration

```

```

22 cat > csr/${NODE_NAME}.cnf <<EOF
23 [req]
24 distinguished_name = req_distinguished_name
25 req_extensions     = v3_req
26 prompt             = no
27
28 [req_distinguished_name]
29 C = US
30 ST = California
31 L = San Francisco
32 O = MediGuard Health Systems
33 OU = Hospital Network
34 CN = ${NODE_NAME}.mediguard.local
35
36 [v3_req]
37 keyUsage             = keyEncipherment, dataEncipherment
38 extendedKeyUsage     = serverAuth, clientAuth
39 subjectAltName       = @alt_names
40
41 [alt_names]
42 DNS.1 = ${NODE_NAME}.mediguard.local
43 DNS.2 = ${NODE_NAME}
44 IP.1  = ${NODE_IP}
45 EOF
46
47 # Generate CSR
48 echo "Generating CSR for ${NODE_NAME}..."
49 openssl req -new \
50     -key private/${NODE_NAME}.key \
51     -out csr/${NODE_NAME}.csr \
52     -config csr/${NODE_NAME}.cnf
53
54 # Sign certificate with Intermediate CA
55 echo "Signing certificate for ${NODE_NAME}..."
56 openssl ca -config openssl-intermediate.cnf \
57     -extensions server_cert \
58     -days 375 -notext -md sha256 \
59     -in csr/${NODE_NAME}.csr \
60     -out certs/${NODE_NAME}.cert \
61     -batch
62
63 # Create certificate bundle
64 cat certs/${NODE_NAME}.cert certs/ca-chain.crt > certs/${NODE_NAME}-
    bundle.crt
65
66 echo "Certificate generated for ${NODE_NAME}"
67 echo "  Private key: ${PKI_DIR}/private/${NODE_NAME}.key"
68 echo "  Certificate: ${PKI_DIR}/certs/${NODE_NAME}.cert"
69 echo "  Bundle:      ${PKI_DIR}/certs/${NODE_NAME}-bundle.crt"

```

```

1 sudo chmod +x /opt/mediguard-pki/generate-node-cert.sh

```

Generate certificates for all nodes:

```

1 cd /opt/mediguard-pki

```

```
2
3 # Hospital A
4 sudo ./generate-node-cert.sh hospital-a 10.0.0.1
5
6 # Hospital B
7 sudo ./generate-node-cert.sh hospital-b 10.0.0.2
8
9 # ML Server
10 sudo ./generate-node-cert.sh ml-server 10.0.0.3
11
12 # Verify certificates
13 for node in hospital-a hospital-b ml-server; do
14     echo "=== ${node} certificate ==="
15     sudo openssl x509 -noout -subject -issuer -dates \
16         -in certs/${node}.cert
17     echo ""
18 done
```

1.1.7 Distribute Certificates to Nodes

From ml-server, copy certificates to hospital VMs:

```
1 # Copy to hospital-a
2 scp /opt/mediguard-pki/certs/hospital-a.crt \
3     /opt/mediguard-pki/private/hospital-a.key \
4     /opt/mediguard-pki/certs/ca-chain.crt \
5     admin@10.0.0.1:/tmp/
6
7 # Copy to hospital-b
8 scp /opt/mediguard-pki/certs/hospital-b.crt \
9     /opt/mediguard-pki/private/hospital-b.key \
10    /opt/mediguard-pki/certs/ca-chain.crt \
11    admin@10.0.0.2:/tmp/
```

On hospital-a:

```
1 sudo mkdir -p /etc/mediguard/pki
2 sudo mv /tmp/hospital-a.crt /etc/mediguard/pki/
3 sudo mv /tmp/hospital-a.key /etc/mediguard/pki/
4 sudo mv /tmp/ca-chain.crt /etc/mediguard/pki/
5 sudo chmod 600 /etc/mediguard/pki/hospital-a.key
6 sudo chmod 644 /etc/mediguard/pki/*.crt
```

On hospital-b:

```
1 sudo mkdir -p /etc/mediguard/pki
2 sudo mv /tmp/hospital-b.crt /etc/mediguard/pki/
3 sudo mv /tmp/hospital-b.key /etc/mediguard/pki/
4 sudo mv /tmp/ca-chain.crt /etc/mediguard/pki/
5 sudo chmod 600 /etc/mediguard/pki/hospital-b.key
```

```
6 sudo chmod 644 /etc/mediguard/pki/*.cert
```

On ml-server:

```
1 sudo mkdir -p /etc/mediguard/pki
2 sudo cp /opt/mediguard-pki/certs/ml-server.cert /etc/mediguard/
  pki/
3 sudo cp /opt/mediguard-pki/private/ml-server.key /etc/mediguard
  /pki/
4 sudo cp /opt/mediguard-pki/certs/ca-chain.cert /etc/mediguard
  /pki/
5 sudo chmod 600 /etc/mediguard/pki/ml-server.key
6 sudo chmod 644 /etc/mediguard/pki/*.cert
```

1.2 AES-256-GCM Encryption Implementation

1.2.1 Create Encryption Module

Create directory structure on all VMs:

```
1 sudo mkdir -p /opt/mediguard/{crypto,keys,logs}
2 cd /opt/mediguard
```

Create file: /opt/mediguard/crypto/encryption.py

```
1 #!/usr/bin/env python3
2 """
3 MediGuard AES-256-GCM Encryption Module
4 Handles encryption/decryption of patient PII fields
5 """
6
7 import os
8 import base64
9 import json
10 from cryptography.hazmat.primitives.ciphers.aead import AESGCM
11 from cryptography.hazmat.primitives import hashes
12 from cryptography.hazmat.primitives.kdf.pbkdf2 import PBKDF2
13 from cryptography.hazmat.backends import default_backend
14 from datetime import datetime
15 import logging
16
17 # Setup logging
18 logging.basicConfig(
19     filename='/opt/mediguard/logs/encryption.log',
20     level=logging.INFO,
21     format='%(asctime)s - %(levelname)s - %(message)s'
22 )
23
24 class EncryptionManager:
25     """
26     Manages AES-256-GCM encryption for patient data
27     """
28
29     def __init__(self, key_file='/opt/mediguard/keys/master.key'):
30         """
31         Initialize encryption manager
32
33         Args:
34             key_file: Path to master encryption key file
35         """
```

```

35     """
36     self.key_file = key_file
37     self.key = self._load_or_generate_key()
38     self.aesgcm = AESGCM(self.key)
39     logging.info("Encryption manager initialized")
40
41     def _load_or_generate_key(self):
42         """
43         Load existing encryption key or generate new one
44
45         Returns:
46             bytes: 256-bit encryption key
47         """
48         if os.path.exists(self.key_file):
49             logging.info(f"Loading encryption key from {self.key_file}")
50             with open(self.key_file, 'rb') as f:
51                 return f.read()
52         else:
53             logging.info("Generating new encryption key")
54             key = AESGCM.generate_key(bit_length=256)
55
56             # Create keys directory if it doesn't exist
57             os.makedirs(os.path.dirname(self.key_file), exist_ok=True)
58
59             # Save key with restrictive permissions
60             with open(self.key_file, 'wb') as f:
61                 f.write(key)
62             os.chmod(self.key_file, 0o600)
63
64             logging.info(f"Encryption key saved to {self.key_file}")
65             return key
66
67     def encrypt(self, plaintext):
68         """
69         Encrypt plaintext data using AES-256-GCM
70
71         Args:
72             plaintext: String or bytes to encrypt
73
74         Returns:
75             bytes: Nonce (12 bytes) + Ciphertext + Auth Tag
76         """
77         if plaintext is None:
78             return None
79
80         # Convert string to bytes if necessary
81         if isinstance(plaintext, str):
82             plaintext = plaintext.encode('utf-8')
83
84         # Generate unique nonce (number used once)
85         nonce = os.urandom(12) # 96-bit nonce recommended for GCM
86
87         # Encrypt with authenticated encryption
88         ciphertext = self.aesgcm.encrypt(nonce, plaintext, None)
89
90         # Return nonce + ciphertext (nonce needed for decryption)
91         encrypted_data = nonce + ciphertext
92
93         logging.debug(
94             f"Encrypted {len(plaintext)} bytes -> {len(encrypted_data)} bytes"
95         )
96         return encrypted_data
97
98     def decrypt(self, encrypted_data):
99         """
100         Decrypt AES-256-GCM encrypted data
101
102         Args:
103             encrypted_data: bytes (nonce + ciphertext)
104

```

```

105     Returns:
106         str: Decrypted plaintext
107     """
108     if encrypted_data is None:
109         return None
110
111     # Convert memoryview to bytes if necessary
112     if isinstance(encrypted_data, memoryview):
113         encrypted_data = bytes(encrypted_data)
114
115     # Extract nonce and ciphertext
116     nonce = encrypted_data[:12]
117     ciphertext = encrypted_data[12:]
118
119     # Decrypt and verify authentication tag
120     try:
121         plaintext = self.aesgcm.decrypt(nonce, ciphertext, None)
122         return plaintext.decode('utf-8')
123     except Exception as e:
124         logging.error(f"Decryption failed: {e}")
125         raise ValueError(
126             "Decryption failed - data may be corrupted or tampered"
127         )
128
129 def encrypt_field(self, field_name, value):
130     """
131     Encrypt a database field with metadata
132
133     Args:
134         field_name: Name of the field being encrypted
135         value: Value to encrypt
136
137     Returns:
138         bytes: Encrypted data
139     """
140     envelope = {
141         'field': field_name,
142         'value': value,
143         'encrypted_at': datetime.utcnow().isoformat()
144     }
145     envelope_json = json.dumps(envelope)
146     encrypted = self.encrypt(envelope_json)
147
148     logging.info(f"Encrypted field: {field_name}")
149     return encrypted
150
151 def decrypt_field(self, encrypted_data):
152     """
153     Decrypt a database field and extract value
154
155     Args:
156         encrypted_data: Encrypted bytes
157
158     Returns:
159         str: Decrypted field value
160     """
161     decrypted_json = self.decrypt(encrypted_data)
162     envelope = json.loads(decrypted_json)
163
164     logging.info(f"Decrypted field: {envelope['field']}")
165     return envelope['value']
166
167 def rotate_key(self, new_key_file='/opt/mediguard/keys/master-new.key'):
168     """
169     Generate new encryption key for key rotation
170
171     Args:
172         new_key_file: Path to save new key
173
174     Returns:

```

```

175         tuple: (old_key, new_key)
176         """
177         old_key = self.key
178
179         # Generate new key
180         new_key = AESGCM.generate_key(bit_length=256)
181
182         # Save new key
183         with open(new_key_file, 'wb') as f:
184             f.write(new_key)
185         os.chmod(new_key_file, 0o600)
186
187         logging.info(f"New encryption key generated: {new_key_file}")
188         logging.warning(
189             "Key rotation initiated - re-encrypt all data with new key"
190         )
191         return (old_key, new_key)
192
193     def re_encrypt(self, old_encrypted_data, new_key):
194         """
195         Re-encrypt data with new key (for key rotation)
196
197         Args:
198             old_encrypted_data: Data encrypted with old key
199             new_key: New encryption key
200
201         Returns:
202             bytes: Data encrypted with new key
203         """
204         # Decrypt with old key
205         plaintext = self.decrypt(old_encrypted_data)
206
207         # Encrypt with new key
208         new_aesgcm = AESGCM(new_key)
209         nonce = os.urandom(12)
210         ciphertext = new_aesgcm.encrypt(
211             nonce, plaintext.encode('utf-8'), None
212         )
213         return nonce + ciphertext

```

```

1  class KeyVault:
2      """
3      Secure key storage with two-factor access control
4      """
5
6      def __init__(self, vault_file='/opt/mediguard/keys/vault.enc'):
7          self.vault_file = vault_file
8
9      def create_vault(self, passphrase, cert_path):
10         """
11         Create encrypted key vault
12
13         Args:
14             passphrase: User passphrase (factor 1)
15             cert_path: Path to mTLS certificate (factor 2)
16         """
17         # Derive key from passphrase
18         salt = os.urandom(16)
19         kdf = PBKDF2(
20             algorithm=hashes.SHA256(),
21             length=32,
22             salt=salt,
23             iterations=100000,
24             backend=default_backend()
25         )
26         vault_key = kdf.derive(passphrase.encode('utf-8'))
27
28         # Read certificate (factor 2)
29         with open(cert_path, 'rb') as f:
30             cert_data = f.read()

```

```

31
32     # Combine factors
33     combined_key = hashes.Hash(hashes.SHA256(), backend=default_backend())
34     combined_key.update(vault_key)
35     combined_key.update(cert_data)
36     final_key = combined_key.finalize()
37
38     # Encrypt master encryption key with vault key
39     aesgcm = AESGCM(final_key)
40     with open('/opt/mediguard/keys/master.key', 'rb') as f:
41         master_key = f.read()
42
43     nonce = os.urandom(12)
44     encrypted_master_key = aesgcm.encrypt(nonce, master_key, None)
45
46     # Save vault
47     vault_data = {
48         'salt': base64.b64encode(salt).decode('utf-8'),
49         'encrypted_key': base64.b64encode(
50             nonce + encrypted_master_key
51         ).decode('utf-8'),
52         'created_at': datetime.utcnow().isoformat()
53     }
54
55     with open(self.vault_file, 'w') as f:
56         json.dump(vault_data, f, indent=2)
57
58     os.chmod(self.vault_file, 0o600)
59     logging.info(f"Key vault created: {self.vault_file}")
60
61 def unlock_vault(self, passphrase, cert_path):
62     """
63     Unlock vault and retrieve master key
64
65     Args:
66         passphrase: User passphrase
67         cert_path: Path to mTLS certificate
68
69     Returns:
70         bytes: Master encryption key
71     """
72     with open(self.vault_file, 'r') as f:
73         vault_data = json.load(f)
74
75     # Derive key from passphrase
76     salt = base64.b64decode(vault_data['salt'])
77     kdf = PBKDF2(
78         algorithm=hashes.SHA256(),
79         length=32,
80         salt=salt,
81         iterations=100000,
82         backend=default_backend()
83     )
84     vault_key = kdf.derive(passphrase.encode('utf-8'))
85
86     # Read certificate
87     with open(cert_path, 'rb') as f:
88         cert_data = f.read()
89
90     # Combine factors
91     combined_key = hashes.Hash(hashes.SHA256(), backend=default_backend())
92     combined_key.update(vault_key)
93     combined_key.update(cert_data)
94     final_key = combined_key.finalize()
95
96     # Decrypt master key
97     encrypted_data = base64.b64decode(vault_data['encrypted_key'])
98     nonce = encrypted_data[:12]
99     ciphertext = encrypted_data[12:]
100

```



```

101     aesgcm = AESGCM(final_key)
102     master_key = aesgcm.decrypt(nonce, ciphertext, None)
103
104     logging.info("Key vault unlocked successfully")
105     return master_key
106
107
108 def demo_encryption():
109     """
110     Demonstrate encryption functionality
111     """
112     print("=" * 60)
113     print("MediGuard AES-256-GCM Encryption Demo")
114     print("=" * 60)
115
116     # Initialize encryption manager
117     em = EncryptionManager()
118
119     # Test data
120     test_data = {
121         'name': 'John Doe',
122         'ssn': '123-45-6789',
123         'dob': '1985-03-15',
124         'diagnosis': 'Type 2 Diabetes',
125         'glucose': '245 mg/dL'
126     }
127
128     print("\n1. Original Data:")
129     for key, value in test_data.items():
130         print(f"    {key}: {value}")
131
132     # Encrypt
133     print("\n2. Encrypting data...")
134     encrypted_data = {}
135     for key, value in test_data.items():
136         encrypted_data[key] = em.encrypt_field(key, value)
137         print(
138             f"    {key}: "
139             f"{base64.b64encode(encrypted_data[key][:20]).decode()}..."
140             f" (truncated)"
141         )
142
143     # Decrypt
144     print("\n3. Decrypting data...")
145     decrypted_data = {}
146     for key, encrypted in encrypted_data.items():
147         decrypted_data[key] = em.decrypt_field(encrypted)
148         print(f"    {key}: {decrypted_data[key]}")
149
150     # Verify
151     print("\n4. Verification:")
152     if test_data == decrypted_data:
153         print("    [OK] Encryption/Decryption successful!")
154     else:
155         print("    [FAIL] Data mismatch!")
156
157     # Demonstrate tamper detection
158     print("\n5. Tamper Detection Test:")
159     encrypted_name = encrypted_data['name']
160     tampered = bytearray(encrypted_name)
161     tampered[15] ^= 0xFF # Flip bits
162
163     try:
164         em.decrypt(bytes(tampered))
165         print("    [FAIL] Tampering not detected!")
166     except ValueError as e:
167         print(f"    [OK] Tampering detected: {e}")
168
169     print("\n" + "=" * 60)
170

```

```

171
172 if __name__ == '__main__':
173     demo_encryption()

```

Install dependencies and run demo:

```

1 sudo apt install python3-pip -y
2 sudo pip3 install cryptography
3
4 cd /opt/mediguard/crypto
5 sudo python3 encryption.py

```

1.3 Database Integration with Encryption

Create file: /opt/mediguard/crypto/db_crypto.py

```

1  #!/usr/bin/env python3
2  """
3  Database Encryption Integration
4  Encrypts patient data before database insert
5  """
6
7  import psycopg2
8  from encryption import EncryptionManager
9  import logging
10
11  logging.basicConfig(level=logging.INFO)
12
13
14  class SecurePatientDB:
15      """
16      PostgreSQL database wrapper with automatic encryption
17      """
18
19      def __init__(self, db_config):
20          """
21          Initialize secure database connection
22
23          Args:
24              db_config: dict with connection parameters
25          """
26          self.conn = psycopg2.connect(**db_config)
27          self.em = EncryptionManager()
28          logging.info("Secure database connection established")
29
30      def insert_patient(self, patient_data):
31          """
32          Insert patient with encrypted PII
33
34          Args:
35              patient_data: dict containing patient information
36
37          Returns:
38              int: patient_id of inserted record
39          """
40          cursor = self.conn.cursor()
41
42          # Encrypt sensitive fields
43          name_enc = self.em.encrypt(patient_data['name'])
44          ssn_enc = self.em.encrypt(patient_data.get('ssn', ''))
45          dob_enc = self.em.encrypt(patient_data['dob'])
46          address_enc = self.em.encrypt(patient_data.get('address', ''))
47          phone_enc = self.em.encrypt(patient_data.get('phone', ''))
48

```

```

49     # Insert with encrypted data
50     cursor.execute("""
51         INSERT INTO patients (
52             name_encrypted, ssn_encrypted, dob_encrypted,
53             address_encrypted, phone_encrypted,
54             gender, blood_type, hospital_id
55         ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
56         RETURNING patient_id
57     """, (
58         name_enc, ssn_enc, dob_enc,
59         address_enc, phone_enc,
60         patient_data.get('gender', 'Unknown'),
61         patient_data.get('blood_type', 'Unknown'),
62         patient_data['hospital_id']
63     ))
64
65     patient_id = cursor.fetchone()[0]
66     self.conn.commit()
67     cursor.close()
68
69     logging.info(f"Patient inserted with encrypted PII: ID {patient_id}")
70     return patient_id
71
72 def get_patient(self, patient_id, decrypt=True):
73     """
74     Retrieve patient with optional decryption
75
76     Args:
77         patient_id: Patient ID to retrieve
78         decrypt: Whether to decrypt PII fields
79
80     Returns:
81         dict: Patient data
82     """
83     cursor = self.conn.cursor()
84
85     cursor.execute("""
86         SELECT
87             patient_id, name_encrypted, ssn_encrypted,
88             dob_encrypted, address_encrypted, phone_encrypted,
89             gender, blood_type, hospital_id
90         FROM patients
91         WHERE patient_id = %s
92     """, (patient_id,))
93
94     row = cursor.fetchone()
95     cursor.close()
96
97     if not row:
98         return None
99
100     patient = {
101         'patient_id': row[0],
102         'gender': row[6],
103         'blood_type': row[7],
104         'hospital_id': row[8]
105     }
106
107     if decrypt:
108         # Decrypt PII fields in memory only
109         patient['name'] = self.em.decrypt(row[1])
110         patient['ssn'] = self.em.decrypt(row[2]) if row[2] else None
111         patient['dob'] = self.em.decrypt(row[3])
112         patient['address'] = self.em.decrypt(row[4]) if row[4] else None
113         patient['phone'] = self.em.decrypt(row[5]) if row[5] else None
114     else:
115         # Return encrypted (for debugging)
116         patient['name_encrypted'] = row[1]
117         patient['ssn_encrypted'] = row[2]
118         patient['dob_encrypted'] = row[3]

```

```

119
120         return patient
121
122     def update_existing_patients(self):
123         """
124         Re-encrypt existing plaintext patient data
125         (Migration script for Phase 2 data)
126         """
127         cursor = self.conn.cursor()
128
129         # Fetch all patients with plaintext data
130         cursor.execute("""
131             SELECT patient_id,
132                    encode(name_encrypted, 'escape'),
133                    encode(dob_encrypted, 'escape')
134             FROM patients
135             LIMIT 10
136         """)
137
138         patients = cursor.fetchall()
139
140         for patient_id, name_plain, dob_plain in patients:
141             name_enc = self.em.encrypt(name_plain)
142             dob_enc = self.em.encrypt(dob_plain)
143
144             cursor.execute("""
145                 UPDATE patients
146                 SET name_encrypted = %s,
147                     dob_encrypted = %s
148                 WHERE patient_id = %s
149             """, (name_enc, dob_enc, patient_id))
150
151         self.conn.commit()
152         cursor.close()
153
154         logging.info(f"Re-encrypted {len(patients)} patient records")
155
156     def close(self):
157         """Close database connection"""
158         self.conn.close()
159         logging.info("Database connection closed")
160
161
162     def demo_db_encryption():
163         """
164         Demonstrate database encryption
165         """
166         DB_CONFIG = {
167             'dbname': 'mediguard_hospital_a',
168             'user': 'mediguard_admin',
169             'password': 'YourSecurePassword123!',
170             'host': 'localhost',
171             'port': '5432'
172         }
173
174         print("=" * 60)
175         print("Database Encryption Demo")
176         print("=" * 60)
177
178         db = SecurePatientDB(DB_CONFIG)
179
180         # Insert new patient with encryption
181         print("\n1. Inserting patient with encrypted PII...")
182         patient_data = {
183             'name': 'Jane Smith',
184             'ssn': '987-65-4321',
185             'dob': '1990-07-22',
186             'address': '123 Main St, San Francisco, CA',
187             'phone': '415-555-1234',
188             'gender': 'F',

```

```

189         'blood_type': 'A+',
190         'hospital_id': 'HOSP-A'
191     }
192
193     patient_id = db.insert_patient(patient_data)
194     print(f"    [OK] Patient inserted: ID {patient_id}")
195
196     # Retrieve with decryption
197     print("\n2. Retrieving patient (decrypted)...")
198     patient_decrypted = db.get_patient(patient_id, decrypt=True)
199     print(f"    Name: {patient_decrypted['name']}")
200     print(f"    DOB: {patient_decrypted['dob']}")
201     print(f"    SSN: {patient_decrypted['ssn']}")
202
203     # Retrieve without decryption (show encrypted)
204     print("\n3. Retrieving patient (encrypted - as stored in DB)...")
205     patient_encrypted = db.get_patient(patient_id, decrypt=False)
206     print(
207         f"    Name (encrypted): "
208         f"{patient_encrypted['name_encrypted'][:30]}... (truncated)"
209     )
210
211     # Demonstrate database admin cannot read PII
212     print("\n4. Database Admin View (direct SQL query):")
213     print("    If a DBA runs: SELECT name_encrypted FROM patients LIMIT 1")
214     print("    They see: \\x8f3a9d2e1f... (binary gibberish)")
215     print("    [OK] PII protected even from database administrators!")
216
217     db.close()
218     print("\n" + "=" * 60)
219
220
221 if __name__ == '__main__':
222     demo_db_encryption()

```

1.4 Digital Signatures Implementation

Create file: /opt/mediguard/crypto/signatures.py

```

1  #!/usr/bin/env python3
2  """
3  Digital Signature Module using RSA-PSS
4  Ensures data integrity and authenticity
5  """
6
7  from cryptography.hazmat.primitives import hashes, serialization
8  from cryptography.hazmat.primitives.asymmetric import rsa, padding
9  from cryptography.hazmat.backends import default_backend
10 import base64
11 import json
12 from datetime import datetime
13 import logging
14
15 logging.basicConfig(level=logging.INFO)
16
17
18 class SignatureManager:
19     """
20     Manages RSA digital signatures for query results
21     """
22
23     def __init__(self, node_name):
24         """
25         Initialize signature manager
26
27         Args:
28             node_name: Name of this node (e.g., 'hospital-a')

```

```

29         """
30         self.node_name      = node_name
31         self.private_key_path = f'/etc/mediguard/pki/{node_name}.key'
32         self.cert_path      = f'/etc/mediguard/pki/{node_name}.cert'
33
34         # Load private key for signing
35         self.private_key = self._load_private_key()
36
37         # Load public key for verification
38         self.public_key = self.private_key.public_key()
39
40         logging.info(f"Signature manager initialized for {node_name}")
41
42     def _load_private_key(self):
43         """
44         Load RSA private key from PEM file
45
46         Returns:
47             RSAPrivateKey object
48         """
49         with open(self.private_key_path, 'rb') as f:
50             private_key = serialization.load_pem_private_key(
51                 f.read(),
52                 password=None,
53                 backend=default_backend()
54             )
55         return private_key
56
57     def sign_data(self, data):
58         """
59         Sign data with RSA-PSS digital signature
60
61         Args:
62             data: bytes or dict to sign
63
64         Returns:
65             dict with 'data', 'signature', 'signer', 'signed_at'
66         """
67         # Convert dict to JSON bytes if necessary
68         if isinstance(data, dict):
69             data_bytes = json.dumps(data, sort_keys=True).encode('utf-8')
70         elif isinstance(data, str):
71             data_bytes = data.encode('utf-8')
72         else:
73             data_bytes = data
74
75         # Sign with RSA-PSS (Probabilistic Signature Scheme)
76         signature = self.private_key.sign(
77             data_bytes,
78             padding.PSS(
79                 mgf=padding.MGF1(hashes.SHA256()),
80                 salt_length=padding.PSS.MAX_LENGTH
81             ),
82             hashes.SHA256()
83         )
84
85         signed_envelope = {
86             'data':      data if isinstance(data, dict)
87                         else data.decode('utf-8'),
88             'signature': base64.b64encode(signature).decode('utf-8'),
89             'signer':    self.node_name,
90             'signed_at': datetime.utcnow().isoformat()
91         }
92
93         logging.info(f"Data signed by {self.node_name}")
94         return signed_envelope
95
96     def verify_signature(self, signed_envelope, public_key=None):
97         """
98         Verify digital signature

```

```

99
100     Args:
101         signed_envelope: dict with data and signature
102         public_key: Public key to verify with (defaults to self)
103
104     Returns:
105         bool: True if signature valid, False otherwise
106     """
107     if public_key is None:
108         public_key = self.public_key
109
110     data = signed_envelope['data']
111     signature = base64.b64decode(signed_envelope['signature'])
112
113     # Convert data to bytes
114     if isinstance(data, dict):
115         data_bytes = json.dumps(data, sort_keys=True).encode('utf-8')
116     elif isinstance(data, str):
117         data_bytes = data.encode('utf-8')
118     else:
119         data_bytes = data
120
121     # Verify signature
122     try:
123         public_key.verify(
124             signature,
125             data_bytes,
126             padding.PSS(
127                 mgf=padding.MGF1(hashes.SHA256()),
128                 salt_length=padding.PSS.MAX_LENGTH
129             ),
130             hashes.SHA256()
131         )
132         logging.info(
133             f"[OK] Signature verified from {signed_envelope['signer']}"
134         )
135         return True
136     except Exception as e:
137         logging.error(f"[FAIL] Signature verification failed: {e}")
138         return False
139
140 def load_peer_public_key(self, peer_name):
141     """
142     Load public key from peer's certificate
143
144     Args:
145         peer_name: Name of peer node
146
147     Returns:
148         RSAPublicKey object
149     """
150     from cryptography import x509
151
152     peer_cert_path = f'/etc/mediguard/pki/{peer_name}.cert'
153
154     with open(peer_cert_path, 'rb') as f:
155         cert = x509.load_pem_x509_certificate(
156             f.read(), default_backend()
157         )
158     return cert.public_key()
159
160
161 def demo_signatures():
162     """
163     Demonstrate digital signatures
164     """
165     print("=" * 60)
166     print("Digital Signature Demo")
167     print("=" * 60)
168

```

```

169     sm = SignatureManager('hospital-a')
170
171     query_result = {
172         'query':      'SELECT COUNT(*) FROM patients WHERE blood_type = "O+",
173         'result':     245,
174         'hospital_id': 'HOSP-A',
175         'timestamp':  datetime.utcnow().isoformat()
176     }
177
178     print("\n1. Original Query Result:")
179     print(f"    {json.dumps(query_result, indent=2)}")
180
181     print("\n2. Signing query result...")
182     signed_result = sm.sign_data(query_result)
183     print(f"    Signature: {signed_result['signature'][:60]}... (truncated)")
184     print(f"    Signer: {signed_result['signer']}")
185
186     print("\n3. Verifying signature...")
187     is_valid = sm.verify_signature(signed_result)
188     print(f"    [OK] Signature valid: {is_valid}")
189
190     print("\n4. Tampering Detection Test:")
191     tampered_result = signed_result.copy()
192     tampered_result['data']['result'] = 999 # Change result
193
194     is_valid_tampered = sm.verify_signature(tampered_result)
195     print(f"    [OK] Tampered signature rejected: {not is_valid_tampered}")
196
197     print("\n" + "=" * 60)
198
199
200 if __name__ == '__main__':
201     try:
202         demo_signatures()
203     except FileNotFoundError as e:
204         print(f"Error: {e}")
205         print("Run PKI setup first (Section 3.1)")

```

1.5 Zero-Knowledge Proofs Implementation

Create file: /opt/mediguard/crypto/zkp.py

```

1  #!/usr/bin/env python3
2  """
3  Zero-Knowledge Proof Implementation
4  Allows proving statements about data without revealing the data itself
5  """
6
7  import hashlib
8  import json
9  from typing import Any, Tuple
10 import logging
11
12 logging.basicConfig(level=logging.INFO)
13
14
15 class ZeroKnowledgeProof:
16     """
17     Simple Zero-Knowledge Proof implementation.
18     Proves data properties without revealing actual data.
19
20     NOTE: This is a simplified educational implementation.
21     Production systems should use zk-SNARKs libraries
22     such as py_ecc or libsnark.
23     """
24
25     def __init__(self):

```



```

26     """Initialize ZKP system"""
27     self.commitments = {}
28     logging.info("ZKP system initialized")
29
30 def commit(self, value: Any, secret: str = None) -> Tuple[str, str]:
31     """
32     Create cryptographic commitment to a value
33
34     Args:
35         value: Value to commit to
36         secret: Random secret (nonce)
37
38     Returns:
39         tuple: (commitment_hash, secret)
40     """
41     import os
42
43     if secret is None:
44         secret = os.urandom(32).hex()
45
46     # Create commitment: H(value || secret)
47     value_str = str(value)
48     commitment_input = f"{value_str}||{secret}"
49     commitment_hash = hashlib.sha256(
50         commitment_input.encode()
51     ).hexdigest()
52
53     self.commitments[commitment_hash] = {
54         'value': value,
55         'secret': secret
56     }
57
58     logging.info(f"Created commitment: {commitment_hash[:16]}...")
59     return commitment_hash, secret
60
61 def prove_greater_than(
62     self, value: float, threshold: float, secret: str
63 ) -> dict:
64     """
65     Prove that value > threshold WITHOUT revealing actual value
66
67     Args:
68         value: Actual value (kept secret)
69         threshold: Threshold to prove against (public)
70         secret: Secret from commitment
71
72     Returns:
73         dict: Zero-knowledge proof
74     """
75     commitment, _ = self.commit(value, secret)
76     is_greater = value > threshold
77
78     proof_input = f"{commitment}||{threshold}||{is_greater}"
79     proof_hash = hashlib.sha256(proof_input.encode()).hexdigest()
80
81     proof = {
82         'commitment': commitment,
83         'threshold': threshold,
84         'result': is_greater,
85         'proof_hash': proof_hash,
86         'type': 'greater_than'
87     }
88
89     logging.info(
90         f"ZKP: Value {'>' if is_greater else '<='} "
91         f"{threshold} (value not revealed)"
92     )
93     return proof
94
95 def verify_proof(self, proof: dict, commitment: str) -> bool:

```

```

96     """
97     Verify zero-knowledge proof
98
99     Args:
100         proof:      ZKP to verify
101         commitment: Original commitment hash
102
103     Returns:
104         bool: True if proof is valid
105     """
106     expected_input = (
107         f"{proof['commitment']}||{proof['threshold']}||{proof['result']}"
108     )
109     expected_hash = hashlib.sha256(expected_input.encode()).hexdigest()
110
111     is_valid = (
112         proof['proof_hash'] == expected_hash and
113         proof['commitment'] == commitment
114     )
115
116     if is_valid:
117         logging.info("[OK] ZKP verification successful")
118     else:
119         logging.error("[FAIL] ZKP verification failed")
120
121     return is_valid
122
123 def prove_range(
124     self, value: float, min_val: float, max_val: float, secret: str
125 ) -> dict:
126     """
127     Prove that min_val <= value <= max_val without revealing value
128
129     Args:
130         value:      Actual value
131         min_val:    Minimum value
132         max_val:    Maximum value
133         secret:     Commitment secret
134
135     Returns:
136         dict: Range proof
137     """
138     commitment, _ = self.commit(value, secret)
139     in_range      = min_val <= value <= max_val
140
141     proof_input = f"{commitment}||{min_val}||{max_val}||{in_range}"
142     proof_hash  = hashlib.sha256(proof_input.encode()).hexdigest()
143
144     proof = {
145         'commitment': commitment,
146         'min':       min_val,
147         'max':       max_val,
148         'in_range':  in_range,
149         'proof_hash': proof_hash,
150         'type':      'range'
151     }
152
153     logging.info(
154         f"ZKP: Value {'in' if in_range else 'outside'} "
155         f"range [{min_val}, {max_val}]"
156     )
157     return proof
158
159 def prove_equality(
160     self, value1: Any, value2: Any, secret1: str, secret2: str
161 ) -> dict:
162     """
163     Prove two committed values are equal without revealing them
164
165     Args:

```

```

166         value1: First value
167         value2: Second value
168         secret1: Secret for value1
169         secret2: Secret for value2
170
171     Returns:
172         dict: Equality proof
173     """
174     commitment1, _ = self.commit(value1, secret1)
175     commitment2, _ = self.commit(value2, secret2)
176     are_equal = value1 == value2
177
178     proof_input = f"{{commitment1}}|{{commitment2}}|{{are_equal}}"
179     proof_hash = hashlib.sha256(proof_input.encode()).hexdigest()
180
181     proof = {
182         'commitment1': commitment1,
183         'commitment2': commitment2,
184         'equal': are_equal,
185         'proof_hash': proof_hash,
186         'type': 'equality'
187     }
188
189     logging.info(
190         f"ZKP: Values are {'equal' if are_equal else 'not equal'}"
191     )
192     return proof

```

```

1  class MedicalZKP:
2      """
3      Medical-specific Zero-Knowledge Proofs
4      """
5
6      def __init__(self):
7          self.zkp = ZeroKnowledgeProof()
8
9      def prove_abnormal_glucose(
10         self, glucose_value: float, patient_secret: str
11     ) -> dict:
12         """
13         Prove patient has abnormal glucose (>200 mg/dL)
14         without revealing exact value
15
16         Args:
17             glucose_value: Patient's glucose level
18             patient_secret: Secret for this patient
19
20         Returns:
21             dict: ZKP that glucose is abnormal
22         """
23         GLUCOSE_THRESHOLD = 200.0
24
25         proof = self.zkp.prove_greater_than(
26             glucose_value, GLUCOSE_THRESHOLD, patient_secret
27         )
28         proof['medical_interpretation'] = (
29             'Abnormal' if proof['result'] else 'Normal'
30         )
31         return proof
32
33     def prove_age_range(
34         self, age: int, min_age: int, max_age: int, patient_secret: str
35     ) -> dict:
36         """
37         Prove patient is in age range without revealing exact age
38
39         Args:
40             age: Patient's actual age
41             min_age: Minimum age
42             max_age: Maximum age

```

```

43         patient_secret: Secret for this patient
44
45     Returns:
46         dict: Age range proof
47     """
48     proof = self.zkp.prove_range(age, min_age, max_age, patient_secret)
49     proof['age_category'] = (
50         'In Range' if proof['in_range'] else 'Out of Range'
51     )
52     return proof
53
54 def prove_high_risk(
55     self, risk_score: float, patient_secret: str
56 ) -> dict:
57     """
58     Prove patient is high-risk (score > 0.7) without revealing score
59
60     Args:
61         risk_score: Patient risk score (0-1)
62         patient_secret: Secret for this patient
63
64     Returns:
65         dict: High-risk proof
66     """
67     HIGH_RISK_THRESHOLD = 0.7
68
69     proof = self.zkp.prove_greater_than(
70         risk_score, HIGH_RISK_THRESHOLD, patient_secret
71     )
72     proof['risk_category'] = (
73         'High Risk' if proof['result'] else 'Low/Medium Risk'
74     )
75     return proof
76
77
78 def demo_zkp():
79     """
80     Demonstrate Zero-Knowledge Proofs for medical data
81     """
82     print("=" * 70)
83     print("Zero-Knowledge Proof Demo - Medical Data Privacy")
84     print("=" * 70)
85
86     medical_zkp = MedicalZKP()
87
88     # Scenario 1: Abnormal glucose detection
89     print("\n" + "=" * 70)
90     print("Scenario 1: Prove abnormal glucose without revealing exact value")
91     print("=" * 70)
92
93     patient1_glucose = 245.0
94     patient1_secret = "secret123"
95
96     print(f"\nActual glucose value: {patient1_glucose} mg/dL (KEPT SECRET)")
97     print("Threshold: >200 mg/dL")
98
99     proof1 = medical_zkp.prove_abnormal_glucose(
100         patient1_glucose, patient1_secret
101     )
102
103     print("\nZero-Knowledge Proof Result:")
104     print(f" [OK] Patient has abnormal glucose: {proof1['result']}")
105     print(f" [OK] Interpretation: {proof1['medical_interpretation']}")
106     print(f" [OK] Commitment: {proof1['commitment'][:32]}...")
107     print(f"\n -> ML Server learns: 'Abnormal glucose = TRUE'")
108     print(f" -> ML Server does NOT learn: Exact glucose ({patient1_glucose})")
109
110     # Scenario 2: Age range verification
111     print("\n" + "=" * 70)
112     print("Scenario 2: Prove patient in age range without revealing exact age")

```

```

113     print("=" * 70)
114
115     patient2_age      = 45
116     patient2_secret  = "secret456"
117
118     print(f"\nActual age: {patient2_age} years (KEPT SECRET)")
119     print("Age range query: 40-60 years")
120
121     proof2 = medical_zkp.prove_age_range(patient2_age, 40, 60, patient2_secret)
122
123     print("\nZero-Knowledge Proof Result:")
124     print(f"    [OK] Patient in range [40-60]: {proof2['in_range']}")
125     print(f"    [OK] Category: {proof2['age_category']}")
126     print(f"\n    -> ML Server learns:      'Age in range = TRUE'")
127     print(f"    -> ML Server does NOT learn: Exact age ({patient2_age})")
128
129     # Scenario 3: Risk score categorisation
130     print("\n" + "=" * 70)
131     print("Scenario 3: Prove high-risk status without revealing risk score")
132     print("=" * 70)
133
134     patient3_risk      = 0.85
135     patient3_secret    = "secret789"
136
137     print(f"\nActual risk score: {patient3_risk} (KEPT SECRET)")
138     print("High-risk threshold: >0.7")
139
140     proof3 = medical_zkp.prove_high_risk(patient3_risk, patient3_secret)
141
142     print("\nZero-Knowledge Proof Result:")
143     print(f"    [OK] Patient is high-risk: {proof3['result']}")
144     print(f"    [OK] Category: {proof3['risk_category']}")
145     print(f"\n    -> ML Server learns:      'High risk = TRUE'")
146     print(
147         f"    -> ML Server does NOT learn: Exact risk score ({patient3_risk})"
148     )
149
150     # Privacy Summary
151     print("\n" + "=" * 70)
152     print("PRIVACY GUARANTEE SUMMARY")
153     print("=" * 70)
154     print("""
155 In traditional ML:
156 [X] Hospital sends: glucose=245, age=45, risk=0.85
157 [X] ML server sees ALL raw values
158 [X] Privacy violation!
159
160 With Zero-Knowledge Proofs:
161 [OK] Hospital sends: ZKP(glucose>200)=TRUE, ZKP(age in [40,60])=TRUE
162 [OK] ML server learns ONLY boolean facts
163 [OK] ML server NEVER sees actual values
164 [OK] Full privacy preserved!
165
166 This is the SAME technology used by:
167 - Zcash cryptocurrency (private transactions)
168 - Ethereum Layer 2 (zkRollups)
169 - Google/IBM healthcare AI research
170 """)
171     print("=" * 70)
172
173
174 if __name__ == '__main__':
175     demo_zkp()

```

Install dependencies and run demos:

```

1  sudo pip3 install cryptography
2

```

```

3  # Run encryption demo
4  cd /opt/mediguard/crypto
5  sudo python3 encryption.py
6
7  # Run digital signature demo (requires PKI setup)
8  sudo python3 signatures.py
9
10 # Run zero-knowledge proof demo
11 sudo python3 zkp.py

```

1.6 Automated Key Rotation

Create file: /opt/mediguard/crypto/key_rotation.sh

```

1  #!/bin/bash
2  # Automated Key Rotation Script
3  # Runs monthly via cron job
4
5  LOG_FILE="/opt/mediguard/logs/key_rotation.log"
6  KEY_DIR="/opt/mediguard/keys"
7  BACKUP_DIR="/opt/mediguard/keys/archive"
8  DATE_STAMP=$(date +%Y%m%d_%H%M%S)
9
10 # Create backup directory
11 mkdir -p $BACKUP_DIR
12
13 log_message() {
14     echo "[$(date +%Y-%m-%d %H:%M:%S)] $1" | tee -a $LOG_FILE
15 }
16
17 log_message "=== Key Rotation Started ==="
18
19 # Backup current key
20 if [ -f "$KEY_DIR/master.key" ]; then
21     log_message "Backing up current master key..."
22     cp $KEY_DIR/master.key ${BACKUP_DIR}/master_${DATE_STAMP}.key
23     chmod 400 ${BACKUP_DIR}/master_${DATE_STAMP}.key
24     log_message "Backup created: master_${DATE_STAMP}.key"
25 fi
26
27 # Generate new key
28 log_message "Generating new encryption key..."
29 python3 -c "
30 import sys
31 sys.path.insert(0, '/opt/mediguard/crypto')
32 from encryption import EncryptionManager
33
34 em = EncryptionManager()
35 old_key, new_key = em.rotate_key()
36 print('New key generated')
37 "
38
39 if [ $? -eq 0 ]; then

```

```

40     log_message "New key generated successfully"
41
42     # TODO: Re-encrypt database (production would have migration
        script)
43     log_message "WARNING: Database re-encryption required"
44     log_message "    Run: python3 /opt/mediguard/crypto/reencrypt_db.
        py"
45
46     # Update key file
47     mv $KEY_DIR/master-new.key $KEY_DIR/master.key
48     chmod 600 $KEY_DIR/master.key
49
50     log_message "Key rotation completed successfully"
51 else
52     log_message "Key rotation FAILED"
53     exit 1
54 fi
55
56 # Delete old backups (keep last 12 months)
57 log_message "Cleaning up old key backups..."
58 find $BACKUP_DIR -name "master_*.key" -mtime +365 -delete
59 log_message "Old backups cleaned"
60
61 log_message "=== Key Rotation Completed ==="

```

Make executable and schedule:

```

1  sudo chmod +x /opt/mediguard/crypto/key_rotation.sh
2
3  # Add to cron (run on 1st of every month at 2 AM)
4  sudo crontab -e

```

Add the following line to the crontab:

```
0 2 1 * * /opt/mediguard/crypto/key_rotation.sh
```

1.7 Phase 3 Verification Checklist

Run these tests to verify the cryptography layer:

```

1  # Test 1: Verify PKI certificates
2  for node in hospital-a hospital-b ml-server; do
3      echo "Testing ${node} certificate..."
4      openssl verify \
5          -CAfile /opt/mediguard-pki/certs/ca-chain.crt \
6          /opt/mediguard-pki/certs/${node}.crt
7  done
8
9  # Test 2: Verify encryption/decryption
10 cd /opt/mediguard/crypto
11 sudo python3 encryption.py
12

```

```

13  # Test 3: Verify digital signatures
14  sudo python3 signatures.py
15
16  # Test 4: Verify zero-knowledge proofs
17  sudo python3 zkp.py
18
19  # Test 5: Check key file permissions
20  ls -la /opt/mediguard/keys/
21  # Should show: -rw----- (600) for key files
22
23  # Test 6: Verify encrypted data in database
24  sudo -u postgres psql -d mediguard_hospital_a -c \
25    "SELECT patient_id,
26       substring(name_encrypted::text, 1, 50)
27    FROM patients LIMIT 3;"
28  # Should show binary/hex data, NOT readable names

```

Success Criteria

- ☐ Root CA and Intermediate CA certificates created
- ☐ Node certificates signed and distributed to all VMs
- ☐ All certificate verifications pass (`openssl verify`)
- ☐ AES-256-GCM encryption and decryption works correctly
- ☐ Digital signatures verify correctly and detect tampering
- ☐ Zero-knowledge proofs demonstrate privacy guarantees
- ☐ Key rotation script executes without errors
- ☐ Database PII fields contain only encrypted binary data

1.8 Troubleshooting

Problem	Solution
Certificate verification fails	Check <code>ca-chain.crt</code> includes both root and intermediate CA
Encryption demo fails	Verify cryptography library installed: <code>pip3 list grep cryptography</code>
Permission denied on key files	Run: <code>sudo chmod 600 /opt/mediguard/keys/*.key</code>
Database shows plaintext	Re-run encryption migration: <code>python3 db_crypto.py</code>

Table 1.1: Common Cryptography Issues and Solutions